Student Task & Assignment Tracker

Professional Software Engineering Project Report

Author: Antigravity AI Architect

Date: June 30, 2026

Version: 1.0.0 (Production-Ready)

Architecture: MVC Layered + Vanilla SPA

1. Abstract

This report presents the system architecture, design specifications, and implementation guidelines for the Student Task & Assignment Tracker. The application is designed to address the scheduling, prioritizing, and reminder management challenges faced by students in managing multi-course workloads. Built using a modern, lightweight tech stack—HTML5, CSS3, Vanilla ES6 JavaScript on the frontend, and Node.js with Express and MongoDB on the backend—this project implements 100% functional modules without third-party frameworks. Key architectural highlights include client-side hash routing, reactive state manager, secure JWT + cookie-based session rotations, schema-validated MongoDB aggregates, background reminder scheduler timers, and deep-level input sanitizations preventing NoSQL injection and Cross-Site Scripting (XSS).

2. Objectives & Scope

2.1 Problem Statement

Modern academic workloads demand concurrent tracking of assignments, quizzes, group projects, and personal deadlines. Relying on paper logs or generic calendars leads to fragmented priorities and missed deadlines. Existing options are often overly complex or require expensive subscriptions. This project creates a free, responsive, student-centric tracker that automates urgencies and triggers reminders before deadlines.

2.2 Goals & Key Accomplishments

- Task Urgency Priority Matrix: Color-coded low, medium, and high categorization.
- Automated Background Reminder Scheduler: 30-second interval checking of scheduled alerts.
- Comprehensive Audit Trail: Capturing security actions and database modification events automatically.
- User Administration Portal: Grid viewing of system accounts, role toggles, and system-wide audits.
- Dynamic Calendar View: Monthly calendar grid mapping tasks to cell items with quick-add capabilities.

3. Technology Stack

The project strictly follows the requested tech boundaries, avoiding bulky SPA frameworks:

- Frontend: Pure HTML5, Vanilla CSS3 (Slate theme tokens), Vanilla JS (Hash Router, Publisher State).
- Backend: Node.js (v20+ Runtime Engine), Express.js MVC routes mapping.
- Database: MongoDB Community Server, Mongoose ODM (Schemas, custom validators, composite indexes).
- Security: Helmet HTTP wrappers, CORS controls, Rate Limiting, Mongo Sanitize, XSS sanitizers.

4. System Architecture

The application utilizes a classic Model-View-Controller (MVC) architecture on the backend, integrated with a reactive Single-Page Application (SPA) on the frontend. The system is split into three layers:

1. Presentation Layer (Client): Handles interface layouts, UI event actions, form checks, and dynamic HTML injection container.

2. Logical Application Layer (Express Controllers): Validates queries, runs user authorization checking, translates database schema results.

3. Data Layer (MongoDB/Mongoose): Persists schemas (User, Task, Category, Notification, Reminder, AuditLog) and executes high-performance aggregation pipelines for dashboard metrics.

5. Testing Methodology & Verification

Verification was performed inside the codebase using Jest and Supertest. All tests execute on a dedicated test database (student_tracker_test) to avoid polluting active data. The test suite covers:

- Authentication testing: Register validations, JWT login checks, expired login tokens block, security roles check.

- Tasks testing: Create tasks, sync background reminders, filter queries, toggle statuses, bulk complete.

- Security testing: Helmet header sniff, custom XSS nested HTML script escapes, Mongo query sanitization checks.

6. Challenges & Conclusion

The Student Task & Assignment Tracker demonstrates that a highly premium, secure, and production-ready application can be engineered efficiently using vanilla technologies. Bypassing massive frontend frameworks significantly reduced load times and simplified routing, while custom middleware modules successfully established robust security protocols (XSS/injection protection) comparable to framework defaults. This system provides a stable foundation for student performance tracking.